built, looks for a project file in the current folder, works out where the corresponding .dlls will be (in the bin folder), and finally runs your app.

In production, you don't need any of this extra work. Your app is already built; it only needs to be run. The `dotnet <dll>` syntax does this alone, so your app starts much faster.

NOTE The `dotnet` command used to run your published application is part of the .NET Runtime. The (identically named) `dotnet` command used to build and run your application during development is part of the .NET software development kit (SDK).

Framework-dependent deployments vs. self-contained deployments

.NET Core applications can be deployed in two ways: runtime-dependent deployments (RDD) and self-contained deployments (SCD).

By default, you'll use an RDD. This relies on the .NET 7 runtime being installed on the target machine that runs your published app, but you can run your app on any platform—Windows, Linux, or macOS—without having to recompile.

By contrast, an SCD contains all the code required to run your app, so the target machine doesn't need to have .NET 7 installed. Instead, publishing your app packages up the .NET 7 runtime with your app's code and libraries.

Each approach has its pros and cons, but in most cases I tend to create RDDs. The final size of RDDs is much smaller, as they contain only your app code instead of the whole .NET 7 framework, which SCDs contain. Also, you can deploy your RDD apps to any platform, whereas SCDs must be compiled specifically for the target machine's operating system, such as Windows 10 64-bit or Red Hat Enterprise Linux 64-bit.

That said, SCDs are excellent for isolating your application from dependencies on the hosting machine. SCDs don't rely on the version of .NET installed on a hosting provider, so you can